⏱ DAY 1: COMPLETE 6-HOUR MASTER TRAINER PLAYBOOK

GLA University — High Performance Computing & Heterogeneous AI Systems

Course Code: GLA-HPC-BCSG701 | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Daily Duration:** 6 Full Hours (Session 1: Morning 3h + Session 2: Afternoon 3h)
Official Syllabus Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)
Day 1 Complete Terminology Glossary: `00_DAY_01_MASTER_TERMINOLOGY_AND_CONCEPT_GLOSSARY.md`
Day 1 Hands-On Lab Runbook: `02_DAY_01_HANDS_ON_LAB_AND_COMMANDS.md`

🎯 1. Official Syllabus Scope (Page 2 of PDF — Strictly Preserved)

The official curriculum specifies **strictly 7 topics for Day 1**:

1. **Introduction to HPC**
2. **Intel HPC architecture (Xeon, GPUs, AI accelerators)**
3. **Linux basics: File system, commands, environment setup**
4. **Shell scripting for automation**
5. **Secured Shell (SSH)**
6. **Intel oneAPI toolkit**
7. **Introduction to Intel HPC Toolkit**

> 📌 **IMPORTANT**
> **Curriculum Boundary Discipline:** Topics like **OpenMP**, **MPI cluster communications**, **Machine Learning / oneDAL**, **TensorFlow-MKL**, **Slurm job queues**, **Horovod**, and **LLMs** belong strictly to Days 2 through 5. Do NOT introduce them on Day 1. Day 1 is dedicated to making 60 students completely fluent in Linux, Shell automation, SSH, and the oneAPI environment across both the morning concept session and afternoon hands-on server lab.

⏰ 2. Complete 6-Hour Dual-Session Master Daily Schedule

```
+-------------------------------------------------------------------------------------+
|                    DAY 1 MASTER 6-HOUR SCHEDULE (09:30 AM - 04:30 PM)               |
+-------------------------------------------------------------------------------------+
| 👨‍🏫 SESSION 1: MORNING THEORY & ARCHITECTURE MASTERCLASS (09:30 AM - 12:30 PM | 3 Hours)  |
|    "Concept Formation → Silicon Architecture → Live Whiteboard Derivations"         |
+-----------------------+-------------------------------------------------------------+
|  09:30 - 10:00 (30m)  | PART 1: Introduction to HPC & Supercomputing Principles     |
|                       |        (Workstation vs Supercomputer, Memory Wall, Slides 1 - 9)  |
+-----------------------+-------------------------------------------------------------+
|  10:00 - 10:45 (45m)  | PART 2: Linux Basics (Filesystem Hierarchy & Environment Setup)   |
|                       |        (VFS Tree, Absolute vs Relative, $PATH, Slide 10)    |
+-----------------------+-------------------------------------------------------------+
|  10:45 - 11:15 (30m)  | PART 3: Shell Scripting for HPC Automation                  |
|                       |        (Shebang, Variables, Streams & hpc_node_audit.sh, Slide 11)|
+-----------------------+-------------------------------------------------------------+
|  11:15 - 11:25 (10m)  | ☕ REJUVENATION BREAK (10 Minutes)                          |
+-----------------------+-------------------------------------------------------------+
|  11:25 - 11:50 (25m)  | PART 4: Secured Shell (SSH) & Cluster Remote Access         |
|                       |        (Asymmetric Crypto, ed25519 Keys, Live `ssh idc-vm` Demo)  |
+-----------------------+-------------------------------------------------------------+
|  11:50 - 12:15 (25m)  | PART 5: Intel oneAPI Toolkit & Intel HPC Toolkit Ecosystem  |
|                       |        (Unified Vision, SYCL Model, setvars.sh, Slides 12 - 14)   |
+-----------------------+-------------------------------------------------------------+
|  12:15 - 12:30 (15m)  | PART 6: Morning Review, 5 Rapid Check Questions & Lunch Teaser    |
+-------------------------------------------------------------------------------------+
| 🍱 LUNCH & INTERMISSION BREAK (12:30 PM - 01:30 PM | 60 Minutes)                    |
|    Cluster node preflight ping, student account credentials verification            |
+-------------------------------------------------------------------------------------+
| 💻 SESSION 2: AFTERNOON HANDS-ON SUPERCOMPUTER LAB (01:30 PM - 04:30 PM | 3 Hours)  |
|    "Hands-On Server Labs → Shell Automation Drill → oneAPI Sandbox → 25 MCQs"       |
+-----------------------+-------------------------------------------------------------+
|  01:30 - 02:00 (30m)  | STAGE 0 & 1: System Readiness & Hardware Introspection      |
|                       |        (lscpu, AVX-512 / AMX detection, Cache Hierarchy probing)  |
+-----------------------+-------------------------------------------------------------+
|  02:00 - 02:45 (45m)  | STAGE 2 & 3: Directory Tree Creation, Heredocs & Permission Drill |
|                       |        (mkdir -p, cat << EOF, chmod 755/644/600, export $PATH)   |
+-----------------------+-------------------------------------------------------------+
|  02:45 - 03:30 (45m)  | STAGE 4: Automated Shell Scripting (Writing hpc_node_audit.sh)    |
|                       |        (Live coding, exit codes, output redirection, execution)   |
+-----------------------+-------------------------------------------------------------+
|  03:30 - 04:00 (30m)  | STAGE 5 & 6: SSH Key Generation & oneAPI Environment Verification |
|                       |        (ssh-keygen -t ed25519, source setvars.sh, sycl-ls check)  |
+-----------------------+-------------------------------------------------------------+
|  04:00 - 04:30 (30m)  | ASSESSMENT & WRAP-UP: 25 Evaluated MCQs & Checklist Sign-Off      |
|                       |        (22-Point Mastery Checklist, 25 MCQs, Day 2 Preview)       |
+-------------------------------------------------------------------------------------+
```

🎙️ 3. Line-by-Line Minute-by-Minute Trainer Delivery Script

PART 1: Introduction to HPC & Intel Architecture (00:00 - 00:30 | 30 Mins)

Slides Mapped: 1 to 9 in `Day-01_Master_Presentation.pptx`

[00:00 - 00:10] Welcome, Program Pedagogy & Scope (Slides 1 - 6)

- **What Trainer Shivam Says (Hinglish/English):**

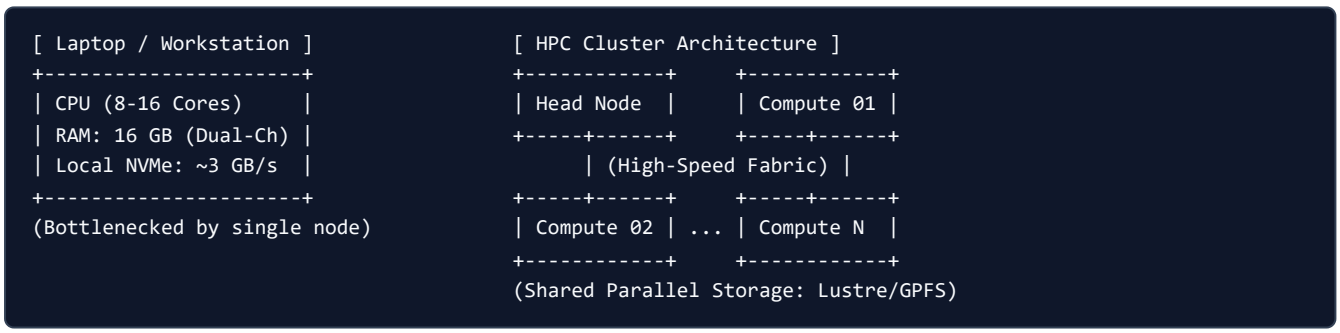
> "Good morning everyone! Welcome to Day 1 of our intensive High Performance Computing and Heterogeneous AI Systems training at GLA University. Over the next 5 days, we are bridging the gap between writing simple code on personal laptops and engineering high-throughput cluster-scale applications. Today is Day 1. Our entire objective today is to build an unbreakable foundation: understanding why HPC exists, how Intel Xeon CPUs, GPUs, and Habana Gaudi accelerators differ, mastering Linux command line, automating tasks with Shell scripts, connecting securely over SSH, and initializing the Intel oneAPI development environment. Let's start with the fundamental question: Why can't we just run big AI models on our laptops?"

[00:10 - 00:20] Topic 1: Introduction to HPC (Slides 7 - 8)

- **Concept:** HPC vs Ordinary Computing.
- **What Trainer Shivam Says:**

> "Look at your laptop right now. If you train a model on 10,000 rows, it works. But if you have 500 million rows or a 70-Billion parameter LLM, what happens? First, your RAM runs out. Second, your memory bandwidth is choked. A typical laptop has 2 memory channels giving around 50 to 80 GB/s bandwidth. An enterprise Dual-Socket Intel Xeon Scalable server has 16 DDR5 memory channels giving over 600 GB/s bandwidth! In HPC, we don't just buy a bigger laptop — we connect dozens of enterprise compute nodes using high-speed interconnects (InfiniBand or 100GbE RoCE) and shared parallel filesystems. HPC combines compute power to solve problems that are computationally impossible on single workstations."

- **Whiteboard Sketch 1 (Draw on board):**



[00:20 - 00:30] Topic 2: Intel HPC Architecture (Slide 9)

- **Concept:** CPU vs GPU vs AI Accelerators.
- **What Trainer Shivam Says:**

> "Modern supercomputers are heterogeneous. There is no single chip that is best at everything. Intel's architecture divides workloads across three distinct silicon engines:
>
> 1. **Intel Xeon Scalable CPUs (Sapphire Rapids / Emerald Rapids):**
>   - Few, very heavy, high-clock cores (32 to 64 cores per socket).
>   - Built for low-latency sequential logic, OS orchestration, data preprocessing, and high-precision FP64 math.
>   - Secret Weapon: **Intel AMX (Advanced Matrix Extensions)** — built-in hardware matrix acceleration on the CPU die!
> 2. **Intel Data Center GPUs (Max Series / Ponte Vecchio):**
>   - Thousands of lightweight SIMD vector cores executing millions of threads in parallel.
>   - Built for massive FP32/FP16 matrix multiplication and computer vision.
> 3. **AI Accelerators (Habana Gaudi 2 / Gaudi 3):**
>   - Built from the silicon up purely for AI tensors and Transformers.
>   - Features 24 Tensor Processor Cores (TPCs) and integrated 24x 100GbE RoCE networking directly on-die! Remember this rule: Xeon orchestrates and preprocesses; GPUs accelerate graphics and general matrix math; Gaudi accelerates massive LLM training."

PART 2: Linux Basics (00:30 - 01:15 | 45 Mins)

Slide Mapped: 10 in `Day-01_Master_Presentation.pptx` | *Colab Cells: 1 to 6*

[00:30 - 00:45] Filesystem Hierarchy & Paths

- **What Trainer Shivam Says:**

> "Open your Google Colab notebook right now. Notice: Google Colab is running on an Ubuntu Linux container! In Windows, you have drive letters: `C:\`, `D:\`. In Linux, there are no drive letters. **Everything is a tree starting at the root** `/`. And the core rule of UNIX is: *Everything is a file!* Let's understand paths:
>
> - **Absolute Path:** Starts from the root `/`. Example: `/home/user/project/script.sh`. Unambiguous from anywhere.
> - **Relative Path:** Starts from where you currently are (Current Working Directory). Example: `./script.sh` or `../data/file.csv`. Let's run `pwd` (Print Working Directory) and `ls -la` in our notebook."

[00:45 - 01:00] Core Linux Commands & Practice

- **Commands Taught:** `pwd`, `cd`, `ls -la`, `mkdir -p`, `touch`, `cp`, `mv`, `rm`, `cat`, `head`, `tail`, `grep`.
- **Live Hands-on Sequence:**

```
# 1. Check working directory
pwd

# 2. Create structured directories
mkdir -p ~/hpc_workspace/day01/{scripts,data,logs}

# 3. Create a test configuration file
cat << 'EOF' > ~/hpc_workspace/day01/scripts/config.env
CLUSTER_NAME="GLA_Intel_HPC"
NODE_COUNT=4
MAX_THREADS=32
EOF

# 4. View and filter contents
cat ~/hpc_workspace/day01/scripts/config.env
grep "NODE_COUNT" ~/hpc_workspace/day01/scripts/config.env
```

- **Trainer Gotcha Warning:**

> "Caution: In Linux, `rm` is permanent! There is no Recycle Bin or Trash can on a remote HPC terminal. Once deleted, your file is gone."

[01:00 - 01:15] Environment Setup & `$PATH`

- **What Trainer Shivam Says:**

> "How does Linux know what to do when you type `python` or `gcc`? It searches the `$PATH` environment variable! An environment variable is a dynamic named value stored in system memory. Let's see our current path: `echo $PATH`. Notice it is a colon-separated list of directories: `/usr/local/bin:/usr/bin:/bin`. Linux scans these folders from left to right. If we install Intel oneAPI in `/opt/intel/oneapi`, our system won't find `icpx` until we add it to `$PATH`! This is why environment setup is critical."

PART 3: Shell Scripting for Automation (01:15 - 01:45 | 30 Mins)

Slide Mapped: 11 in `Day-01_Master_Presentation.pptx` | Colab Cells: 7 to 9

[01:15 - 01:30] The Shebang, Variables, and Permissions

- **What Trainer Shivam Says:**

> "Why do supercomputing engineers write Shell scripts instead of typing commands manually? Because in HPC, workflows have 20 steps: loading modules, allocating nodes, copying weights, launching jobs, and collecting logs. Doing that manually causes human errors. A Shell script guarantees 100% reproducible execution. Every bash script starts with line 1: `#!/bin/bash`. This is called the **Shebang**. It tells the operating system kernel which interpreter to use. Second rule: `VAR="value"` — NEVER put spaces around `=` in bash! Third rule: `chmod +x script.sh` — by default, newly created files do NOT have execute permissions. You must add `+x`."

[01:30 - 01:45] Hands-On: Writing the Node Audit Script

- **What Students Run in Colab (Cell 8):**

```
cat << 'EOF' > hpc_node_audit.sh
#!/bin/bash
set -e
echo "========================================="
echo "  HPC NODE HARDWARE AUDIT REPORT"
echo "  Timestamp: $(date)"
echo "========================================="
echo "1. Hostname:      $(hostname)"
echo "2. Kernel:        $(uname -r)"
echo "3. CPU Model:     $(lscpu | grep 'Model name:' | sed 's/Model name:[ \t]*//')"
echo "4. Total Cores:   $(nproc)"
echo "5. Total Memory:  $(free -h | awk '/^Mem:/ {print $2}')"
echo "========================================="
EOF

chmod +x hpc_node_audit.sh
./hpc_node_audit.sh
```

- **Trainer Check:** Ensure all 60 students execute this script and see their Colab CPU and RAM specs on screen!

☕ BREAK (01:45 - 01:55 | 10 Minutes)

Students stretch, check Colab status, and prepare for the remote cluster demonstration.

PART 4: Secured Shell (SSH) & Remote Access (01:55 - 02:20 | 25 Mins)

Slide Mapped: 11 in `Day-01_Master_Presentation.pptx` | *Live Terminal Demo*

[01:55 - 02:10] Local vs Remote, SSH Architecture & Key Pairs

- **What Trainer Shivam Says:**

> "Nobody sits in an air-conditioned datacenter with a monitor plugged into a supercomputer. Every supercomputer in the world is managed remotely over **SSH (Secure Shell)** on TCP Port 22. How does SSH work?
>
> - **Password Authentication:** Slow, prone to brute-force attacks, cannot be automated easily.
> - **Key-Based Authentication (Asymmetric Cryptography):**
>   1. You generate an `ed25519` key pair on your machine using `ssh-keygen`.
>   2. You get two keys: `id_ed25519` (Private Key — NEVER share!) and `id_ed25519.pub` (Public Key).
>   3. The public key is placed in the remote server's `~/.ssh/authorized_keys`.
>   4. When connecting, the server encrypts a challenge with your public key; your client proves identity using your private key. Fast, 100% secure, passwordless!"

[02:10 - 02:20] Live Trainer Demo: Connecting to Enterprise Intel Cluster (`idc-vm`)

- **Trainer Action (Show on Projector):**

```
ssh idc-vm
```

- **Trainer Script:**

> "Watch my terminal screen on the projector right now. I just typed `ssh idc-vm`. I am now connected live via Tailscale to our bare-metal Intel Developer Cloud node running a real **4th Gen Intel Xeon Gold 6442Y (Sapphire Rapids)** processor! Let's verify: `lscpu | grep -E 'Model name|avx512|amx'`. Notice the flags: `avx512_bf16` and `amx_tile`. This is genuine enterprise supercomputing hardware."

PART 5: Intel oneAPI Toolkit & HPC Toolkit (02:20 - 02:45 | 25 Mins)

Slides Mapped: 12 - 14 in `Day-01_Master_Presentation.pptx` | Colab Cells: 10 - 15

[02:20 - 02:35] Topic 6: Intel oneAPI Toolkit (Slide 12)

- **Concept:** Cross-architecture programming model.
- **What Trainer Shivam Says:**

> "Historically, if you wrote GPU code for NVIDIA, you had to use proprietary CUDA. If you wrote for Intel or AMD, you had to rewrite everything in OpenCL. **Intel oneAPI is an open, standards-based, cross-architecture programming model**. It uses **SYCL** (Standard C++ heterogeneous abstraction). You write single-source modern C++ code, and oneAPI compiles it to run on CPUs, GPUs, and FPGAs! To activate oneAPI in any terminal, you run: `source /opt/intel/oneapi/setvars.sh` This script automatically configures `$PATH`, `$LD_LIBRARY_PATH`, and compiler environment variables."

[02:35 - 02:45] Topic 7: Introduction to Intel HPC Toolkit (Slide 13 - 14)

- **Concept:** Base Toolkit vs HPC Toolkit.
- **What Trainer Shivam Says:**

> "What is the difference between the Base oneAPI Toolkit and the HPC Toolkit?
> - **Base Toolkit:** Core DPC++/SYCL compiler, oneMKL (Math Kernel Library), oneDAL (Data Analytics), and oneDNN (Deep Learning).
> - **HPC Toolkit:** Layers on top of Base Toolkit for cluster supercomputing. It includes:
>   1. `icx` & `icpx` (Intel C/C++ Compilers with advanced vectorization)
>   2. `ifx` (Intel Fortran Compiler for legacy scientific simulations)
>   3. `Intel MPI Library` (Multi-fabric message passing for multi-node communication)
>   4. `Intel Inspector` (Memory and thread error debugger) Tomorrow on Day 2, we will use these exact tools to write parallel OpenMP and MPI programs!"

PART 6: Morning Review, 5 Rapid Check Questions & Lunch Teaser (12:15 - 12:30 | 15 Mins)

Slide Mapped: 15 in `Day-01_Master_Presentation.pptx`

[12:15 - 12:25] 5-Question Quick Classroom Check

1. *Q1: What does HPC stand for and why do ordinary laptops fail on large AI models?*
   Answer: High Performance Computing. Laptops fail due to memory wall (limited DDR channels/bandwidth) and single-node compute bottlenecks.
2. *Q2: Name the 3 hardware pillars of Intel HPC architecture and their primary roles.*
   Answer: Intel Xeon CPUs (orchestration/preprocessing), Intel GPUs (massive parallel matrix/vector math), Habana Gaudi (deep learning/LLM training).

3. *Q3: What is the difference between an absolute path and a relative path in Linux?*
   Answer: Absolute path starts from root `/`; relative path starts from Current Working Directory.
4. *Q4: Why is key-based SSH authentication preferred over passwords in HPC clusters?*
   Answer: Key-based uses asymmetric cryptography (`ed25519`), eliminates brute-force attacks, and enables passwordless automated batch jobs.
5. *Q5: What command initializes the Intel oneAPI development environment?*
   Answer: `source /opt/intel/oneapi/setvars.sh`.

[12:25 - 12:30] Lunch Break Announcement & Afternoon Setup

- **What Trainer Shivam Says:**

> "Excellent work on the theoretical foundation this morning! We have covered the complete architecture of supercomputing, Linux VFS, Shell scripts, SSH, and oneAPI. We now take a 1-hour lunch break until 01:30 PM. When we return at 01:30 PM sharp, we open our Linux terminals and Google Colab notebooks for Session 2: our 3-hour hands-on supercomputing lab where every single student will run hardware introspection, author their own Shell automation scripts, configure SSH keys, and verify the oneAPI environment. Enjoy your lunch!"

🍱 INTERMISSION: Lunch & Cluster Preflight (12:30 PM - 01:30 PM | 60 Mins)

- **Trainer System Checklist during break:**
  1. Verify university lab workstation network connectivity.
  2. Ping the Intel Developer Cloud node (`idc-vm`) to verify Tailscale mesh tunnel status.
  3. Confirm all student Google Colab instances have clean session memory (`free -h`).
  4. Ensure projector displays the terminal prompt for Stage 0 preflight.

💻 4. SESSION 2: Afternoon Hands-On Supercomputer Lab Playbook (01:30 PM - 04:30 PM | 3 Hours)

Reference Runbook: `02_DAY_01_HANDS_ON_LAB_AND_COMMANDS.md`
Notebook Reference: `Day_01_Intel_oneAPI_and_HPC_Toolkit_Master_Lab.ipynb`

```
+-------------------------------------------------------------------------------+
|                  SESSION 2: 3-HOUR HANDS-ON LAB EXECUTION PHASES              |
+-------------------------------------------------------------------------------+
| 01:30 - 02:00 (30m) | Phase 1: Environment Readiness & Silicon Introspection (Stages 0-1)|
| 02:00 - 02:45 (45m) | Phase 2: Linux Tree Hierarchy, Heredocs & Permissions (Stages 2-3) |
| 02:45 - 03:30 (45m) | Phase 3: Shell Automation Drill — Node Audit Script (Stage 4)      |
| 03:30 - 04:00 (30m) | Phase 4: SSH Key-Based Remote Access & oneAPI Setup (Stages 5-6)   |
| 04:00 - 04:30 (30m) | Phase 5: Student 25 MCQs Assessment & 22-Point Checklist Sign-Off  |
+-------------------------------------------------------------------------------+
```

[01:30 - 02:00] Phase 1: Environment Readiness & Silicon Introspection (Stages 0 & 1)

- **Trainer Instruction (To Class):**

> "Welcome back! Open your terminal or the Day 1 Google Colab notebook. Let's start with Stage 0 and Stage 1. We are going to interrogate the physical silicon of our compute host. Run `lscpu`. Do not just glance at it; look for the cache sizes: L1 data, L1 instruction, L2, and shared L3. Now run `lscpu | grep -E 'avx512|amx'`. If you are on an enterprise Xeon Sapphire Rapids node, you will see `amx_tile` and `avx512_bf16`."

- **Trainer Walkabout Check:**
  - Walk down the aisles. Verify that all 60 students have executed `free -h` and `lscpu`.
  - Point out NUMA nodes on the cluster workstations versus unified memory on Colab.

[02:00 - 02:45] Phase 2: Linux Tree Hierarchy, Heredocs & Permissions (Stages 2 & 3)

- **Trainer Instruction (To Class):**

> "Now we execute Stage 2 and Stage 3. In supercomputing, messy filesystems cause catastrophic job failures. We will build a structured workspace using `mkdir -p ~/hpc_day01/{scripts,data,logs,bin}`. Next, we use Heredoc syntax `cat << 'EOF' > cluster_config.env` to write structured configuration files directly from the command line without opening a graphical text editor. Then, test permissions: try running a script with `chmod 644` vs `chmod 755`. See the exact `Permission denied` error and understand octal security bits!"

- **Trainer Common Gotcha Intervention:**
  - When students accidentally omit single quotes in `cat << 'EOF'`, explain that variable expansion happens at heredoc creation rather than script runtime.

[02:45 - 03:30] Phase 3: Shell Automation Drill — Writing `hpc_node_audit.sh` (Stage 4)

- **Trainer Instruction (To Class):**

> "Stage 4 is our first real HPC automation milestone. You are going to write a complete bash script: `hpc_node_audit.sh`. It must gather: Hostname, Linux kernel version, CPU Model, total hardware cores, available RAM, and root filesystem disk space. It must log this data into `~/hpc_day01/logs/audit_$(date +%Y%m%d).log`. Add `set -euo pipefail` at the top so that if any pipeline step fails, the script aborts safely!"

- **Student Milestone Challenge:**
  - Every student must make the script executable (`chmod +x hpc_node_audit.sh`), run it, and show the generated log file using `cat` and `grep`.

[03:30 - 04:00] Phase 4: SSH Key-Based Remote Access & oneAPI Setup (Stages 5 & 6)

- **Trainer Instruction (To Class):**

> "In Stage 5, generate your secure `ed25519` key pair: `ssh-keygen -t ed25519 -N '' -f ~/.ssh/id_ed25519`. Inspect the private key and public key. Notice that `id_ed25519` has permissions `600` (read/write only by owner). Now look at the projector: I will demonstrate copying the public key to a remote server's `~/.ssh/authorized_keys` and logging in passwordless. In Stage 6, we initialize Intel oneAPI: `source /opt/intel/oneapi/setvars.sh` and run `sycl-ls` to inspect all detected CPU and GPU execution targets!"

[04:00 - 04:30] Phase 5: Student 50 MCQs Assessment & 22-Point Checklist Sign-Off

- **Trainer Activity:**
  1. Open `GLA_University_Day_01_Pre_Assessment_50_MCQs.xlsx` and `03_DAY_01_STUDENT_CHEATSHEET_AND_MCQS.md`.

2. Project or administer the 50 Evaluated Pre/In-Class MCQs. Have students submit their responses.
3. Walk through the 22-point Day 1 Mastery Checklist. Verify that every student checks all 22 boxes.
4. **Day 2 Preview Announcement:**

> "Congratulations on completing Day 1! You have utilized 100% of today's 6 hours. You understand HPC architecture, Linux systems, Shell scripts, SSH security, and the Intel oneAPI foundation. Tomorrow on Day 2, we break the serial bottleneck! We will dive into OpenMP multi-core multithreading, MPI distributed cluster communications, and 20x machine learning acceleration with Intel oneDAL. Be here at 09:30 AM sharp!"

🛠️ 5. Top 5 Student Gotchas & 30-Second Rapid Fixes

Issue Encountered	Why It Happened	30-Second Trainer Fix
`bash: ./script.sh: Permission denied`	Created file lacks executable bit	Run `chmod +x script.sh`
`command not found: VAR="val"`	Student put spaces around `=`	Explain: Bash treats spaces as delimiters; use `VAR="val"`
`cd: No such file or directory`	Used relative path from wrong folder	Run `pwd` first, then use absolute path `cd ~/hpc_workspace`
`icpx: command not found`	oneAPI environment not sourced	Run `source /opt/intel/oneapi/setvars.sh`
Colab disconnected / kernel died	Memory exceeded in background	Restart runtime (`Runtime -> Restart session`) and re-run cell

---

🧠 6. The Shivam SEED Pedagogical & Dot-Connecting Blueprint

"Kanthasth Concept Engineering" — How to Make Concepts Live Permanently in Students' Minds

A. The Pre-Assessment 40%–60% Baseline Calibration

- **The Cognitive Principle:** If a 7th-semester CSE student scores 85%+ on a pre-assessment, they dismiss the workshop as trivial ("hume sab aata hai"). If they score under 20%, they suffer learned helplessness ("ye hamare bas ka nahi hai").
- **The Sweet Spot (40% - 60%):**
  - ~25 questions test subjects they already passed (OS, Computer Architecture, Bash, SSH). They feel capable and respected.
  - ~15 questions test Day 1 applied concepts (Amdahl's law math, SYCL work-group division, umask octal calculation). They see the practical relevance.
  - ~10 questions test enterprise Intel silicon (AMX tile register capacity, Gaudi 2 RoCE fabric, NUMA penalties). They experience the exact gap in their knowledge, igniting genuine curiosity for the 6-hour lecture!

B. The In-Class MCQ Socratic Flow (Connecting the Dots)

Do NOT treat MCQs as a detached final exam. Use them interactively throughout the 6 hours:

```
[ Pose Challenge Question ] ⟶ [ Student Thinks & Commits ] ⟶ [ Reveal Physical Mechanism ] ⟶ [ Connect to Nex
```

- **Example Dot-Connecting Chain:**
  1. *Dot 1:* Ask Q05 (Amdahl's Law Speedup limit). *Why can't we speed up infinitely by adding 1000 cores?* (Answer: Serial fraction limits max speedup to 5x).
  2. *Dot 2:* Connect to Hardware: *So how does Intel speed up the parallel part without burning wattage?* (Answer: AVX-512 vectorization and AMX 2D Matrix Tiles).
  3. *Dot 3:* Connect to OS: *Why does this run on Linux servers instead of Windows?* (Answer: VFS, zero-GUI overhead, deterministic process scheduling, signals).
  4. *Dot 4:* Connect to Automation: *Why do we write Shell scripts?* (Answer: Reproducible batch job execution without human error).
  5. *Dot 5:* Connect to Access: *Why SSH keypairs?* (Answer: Passwordless cluster communication across compute nodes).
  6. *Dot 6:* Connect to oneAPI: *Why SYCL?* (Answer: Single-source C++ code running seamlessly across Xeon CPU, Data Center GPU, and Gaudi AI accelerators).

C. Dual-Lab Architecture: Google Colab + Dedicated Linux Servers/VMs

- **Google Colab:** Used for interactive, rapid prototyping, Jupyter visualizations, and instant execution without environment crashes.
- **Dedicated Linux Servers/VMs (e.g., IDC / Intel Developer Cloud):** Used for genuine terminal experience — running `lscpu`, inspecting hardware registers, configuring `~/.ssh/authorized_keys`, and sourcing `oneapi-vars.sh`.

D. Strict 6-Hour Syllabus Boundary Discipline

- Stick rigorously to the 7 Day-1 topics from `Intel & NEC - High Performance Computing.pdf` (Page 2).
- Never allow discussions to wander into Day 2 (OpenMP/MPI distributed algorithms), Day 3 (Horovod / Deep Learning), Day 4 (Slurm sbatch scripts / VTune profiling), or Day 5 (GenAI / OpenVINO).
- This protects both trainer bandwidth and student cognitive load, guaranteeing that every student masters Day 1 completely before moving forward.

E. Twin Invariant: Business Excellence + Student Welfare

- **Institutional / Business Excellence (GLA University & Edulateral):** Impeccable formatting, zero syntax errors, professional 7-column Excel sheets, and clean pedagogical alignment.
- **Student Welfare & Empowerment:** Practical, interview-grade knowledge that prepares students to answer real HPC engineering questions in campus recruitment drives and technical placements.